arXiv:2605.14558v1 [cs.LG] 14 May 2026

Resolving Action Bottleneck: Agentic Reinforcement Learning Informed by Token-Level Energy

Langzhou He[1], **Junyou Zhu**[2,3], **Yue Zhou**[1], **Zhengyao Gu**[1],
Junhua Liu[4], **Wei-Chieh Huang**[1], **Henry Peng Zou**[1], **David Wipf**[5],
Philip S. Yu[1], **Qitian Wu**[6]*

[1]University of Illinois Chicago [2]Potsdam Institute for Climate Impact Research
[3]Technical University of Berlin [4]University of Southern California
[5]University of Hong Kong [6]Broad Institute of MIT and Harvard

`{lhe24,yzhou232,zgu24,whuang80,pzou3,psyu}@uic.edu`
`junyou.zhu@pik-potsdam.de` `jliu2321@usc.edu`
`dwipf@hku.hk` `wuqitian@broadinstitute.org`

Abstract

Agentic reinforcement learning trains large language models using multi-turn trajectories that interleave long reasoning traces with short environment-facing actions. Common policy-gradient methods, such as PPO and GRPO, treat each token in a trajectory equally, leading to uniform credit assignment. In this paper, we critically demonstrate that such uniform credit assignment largely misallocates token-level training signals. From an energy-based modeling perspective, we show that token-level training signals, quantified by their correlations with reward variance of different rollouts sampled from a given prompt, concentrate sharply on action tokens rather than reasoning tokens, even though action tokens account for only a small fraction of the trajectory. We refer to this phenomenon as the Action Bottleneck. Motivated by this observation, we propose an embarrassingly simple token reweighting approach, ACTFOCUS, that downweights gradients on reasoning tokens, along with an additional energy-based redistribution mechanism that further increases the weights on action tokens with higher uncertainty. Across four environments and different model sizes, ACTFOCUS consistently outperforms PPO and GRPO, yielding final-step gains of up to 65.2 and 63.7 percentage points, respectively, without any additional runtime or memory cost.

1 Introduction

As large language models (LLMs) are increasingly deployed as autonomous agents that combine reasoning [8, 9], planning [37], and tool use [31] to solve multi-turn, long-horizon tasks, reinforcement learning (RL) has emerged as the natural post-training framework for sequential decision-making settings [29, 27, 32, 1]. Despite rapid progress with different policy optimization methods [20, 22], training LLM agents with RL remains difficult. One of the core challenges is credit assignment [17, 10], where, in multi-turn trajectories, a sparse episodic reward must be propagated back through thousands of generated tokens, making it difficult to determine which parts of a trajectory should drive learning. The problem is compounded by optimization instability [5], as delayed feedback, long generated traces, and noisy policy updates often lead to degraded late-stage performance or collapse.

Recent evidence shows that multi-turn, underspecified conversations substantially increase outcome variability for LLMs, sharpening the need for reliable credit assignment in agentic RL [11]. In a

---

[*] Corresponding author.

Preprint.

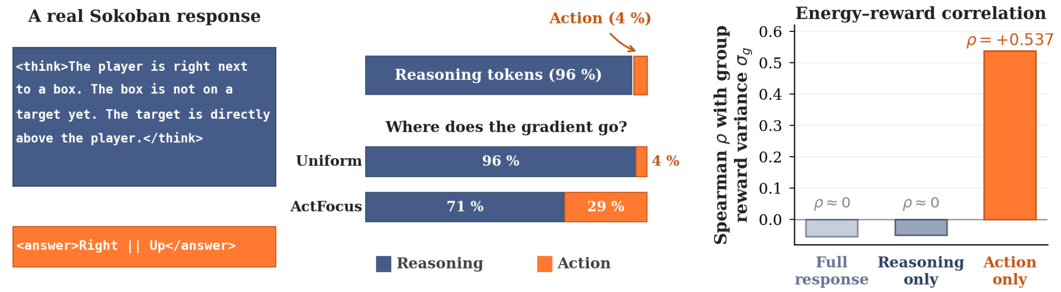


Figure 1: **Action Bottleneck in agentic reinforcement learning.** *Left:* A real response from a trajectory, where reasoning tokens far outnumber action tokens. *Middle:* Action tokens constitute only 4% of model-generated tokens; our token-level reweighting redirects gradient mass towards them. *Right:* Training signal concentrates in action spans. Results are from Sokoban 3B.

typical agentic RL rollout, such outcome variability often hinges on a few critical environment-facing action tokens that drive state transitions, yet these tokens are buried within massive amounts of reasoning tokens in the trajectory [35, 19, 30]. Even though these action tokens are the only parts of the output that directly interact with the environment, standard policy optimization methods treat all generated tokens uniformly [20, 14]. Consequently, gradient updates are spread equally across the whole trajectory, diluting training signals for the critical action tokens that trigger the decision-making [2, 26]. In multi-turn settings, this mismatch compounds with each additional turn: reasoning tokens keep accumulating while the environment is driven by only a few informative actions.

Figure 1 shows that this structural mismatch manifests as token-level signal sparsity in agentic RL. In a representative Sokoban trajectory, action tokens constitute only 4% of the generated output, while reasoning tokens account for the remaining 96%. Under uniform credit assignment, gradient mass therefore follows token count and is dominated by the reasoning majority. However, in this paper, we demonstrate that the training signals follow the opposite pattern. We quantify each token's training signal as its predictive uncertainty under a frozen reference model, measured by the free energy of its next-token predictive distribution.[2] To further evaluate how informative the signal is for training, we hinge it on outcome variability: for each prompt, we calculate the variance of episodic rewards σ_g across multiple rollouts (sampled from the same prompt), which captures how uncertain the current policy is about the prompt's outcome. We find that the mean energy of action tokens is strongly correlated with reward variance (with $\rho = +0.537$), while the mean energy of reasoning tokens or full trajectories, which average over both action and reasoning tokens, is nearly uninformative (with $\rho \approx 0$). **This suggests that informative training signals are concentrated in a small number of action tokens rather than being distributed uniformly across the trajectory.** We refer to this phenomenon as the *Action Bottleneck*.

Motivated by this observation, we propose an energy-based approach to token-level credit assignment in agentic RL. The method is conceptually intuitive and can be decomposed into two aspects: 1) We downweight gradients on reasoning tokens, which is equivalent to increasing the weights on action tokens and helps to redirect gradient mass toward the sparse action span that determines environmental outcomes; 2) We utilize token-level energy to further redistribute the weights across action tokens, which prioritizes action tokens with high energy (i.e., more predictive uncertainty). In this way, we transform our empirical observations into an actionable approach that systematically guides the model to better leverage the informative training signals underlying a few critical tokens in the long trajectory.

We evaluate our approach, ACTFOCUS, on four representative multi-turn agentic environments using Qwen2.5-Instruct [18] models at multiple scales, and train each configuration with both PPO [20] and GRPO [22]. Through these experiments, we find that ACTFOCUS produces consistent performance gains over PPO and GRPO, both of which use uniform credit assignment, with final success-rate improvements of up to 65.2 and 63.7 percentage points, respectively. Beyond absolute performance

---

[2] An ideal indicator for quantifying training signals should reveal where outcome uncertainty concentrates, stay stable across training, and be computationally inexpensive. Energy, used in this work, satisfies all of these criteria; alternatives such as entropy and log-likelihood are compared empirically in Sec. 5.6.

metrics, we observe that ACTFOCUS effectively improves GRPO's training stability by substantially reducing peak-to-final degradation. Our main contributions are summarized as follows:

(i) We analyze token-level attributions in agentic RL training through an energy-based modeling perspective and reveal a counter-intuitive phenomenon: while reasoning tokens dominate in long-horizon multi-turn trajectories, infrequent action tokens contribute the majority of informative training signals. In particular, the uncertainty in action tokens, as measured by the free energy of next-token predictive distributions, is highly correlated with outcome variability.

(ii) We propose a token reweighting mechanism that downweights gradients on reasoning tokens and additionally redistributes the weights on action tokens according to their uncertainty levels as characterized by the free energy. This simple scheme shifts gradient mass towards the most informative action tokens while remaining fully compatible with common policy-gradient algorithms.

(iii) Through extensive experiments across four environments and multiple model scales, ACTFOCUS consistently outperforms PPO and GRPO, with average gains of 17.0 and 34.6 percentage points and final-step gains of up to 65.2 and 63.7 percentage points, respectively. In particular, ACTFOCUS can significantly enhance the training stability of GRPO by mitigating the peak-to-final degradation observed under uniform credit assignment.

2 Related Work

Agentic Reinforcement Learning for LLMs. Reinforcement learning has become a central paradigm for aligning large language models with sequential decision-making objectives beyond single-turn text generation [16]. Recent work extends RL to multi-turn agent settings from different angles. For instance, LOOP [3] formulates interactive digital agents as RL in stateful environments, while RAGEN [29] presents a general framework for multi-turn agent training under stochastic environments. ARLArena [28] systematizes the study of training stability by decomposing policy-gradient design into core dimensions. However, despite these advances, credit assignment in existing agentic RL methods remains relatively under-explored and does not explicitly distinguish between different parts of an agent trajectory, particularly reasoning and action tokens.

Credit Assignment in Agentic RL. A growing line of work argues that uniform credit assignment largely under-represents the token-level training signals of long-horizon agent trajectories. Most existing efforts operate at the step or trajectory level. CARL [23] uses policy entropy as a proxy for action importance and concentrates optimization on critical actions across turns. GiGPO [6] introduces a two-level advantage estimator based on anchor states that reappear across trajectories, enabling step-level relative credit assignment. iStar [13] learns implicit step-wise rewards through trajectory-level preference optimization and combines step-level and trajectory-level signals in policy updates. These methods recognize that credit should vary across an agent trajectory, but they remain coarse in granularity and do not distinguish reasoning tokens from action tokens within each step. Several works in reasoning RL have explored finer-grained reward redistribution and intermediate supervision [25, 4, 15], but these approaches are designed for single-turn reasoning without diving into the reasoning-action disentanglement in multi-turn agent trajectories.

3 Preliminaries

3.1 Multi-Turn Agentic RL

We formulate multi-turn agentic RL as a Markov Decision Process $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R \rangle$ [24], where states $s \in \mathcal{S}$ are interaction histories, actions $a \in \mathcal{A}$ are environment-facing operations available to the agent, the transition $P(s_{k+1} \mid s_k, a_k)$ specifies how the environment responds to an action, and the reward $R(\tau)$ assigns a scalar episodic return based on the terminal state of the trajectory. At each turn $k \in \{1, \dots, K\}$, the policy π_θ observes the current state s_k and generates a structured response:

$$o_k = \texttt{<think>} \dots \texttt{</think>}\ \texttt{<answer>}\ a_k\ \texttt{</answer>}, \tag{1}$$

where the `<think>` block contains the agent's reasoning and the `<answer>` block contains the action a_k. The environment extracts a_k from o_k, transitions to the next state s_{k+1}, and returns a reward $R(\tau)$ at episode termination. The full trajectory is $\tau = (s_1, o_1, s_2, o_2, \dots, s_K, o_K)$, and the RL objective is to maximize expected episodic reward: $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$.

Table 1: Action token fraction in inference-only rollouts before RL training. The imbalance is consistent across environments and model scales.

Env.	**Scale**	**Tokens**	**Think**	**Act**
Sokoban	0.5B	225	90.5%	9.5%
Sokoban	3B	202	96.1%	3.9%
FrozenLake	0.5B	284	95.9%	4.1%
FrozenLake	3B	240	97.4%	2.6%
Sudoku	0.5B	400	84.7%	15.3%
Sudoku	1.5B	397	85.8%	14.2%
WebShop	1.5B	1,951	89.2%	10.8%
WebShop	3B	2,133	92.7%	7.3%

Token-level policy optimization. Each response o_k consists of tokens $(y_{k,1}, \ldots, y_{k,T_k})$ from both the reasoning and action segments. For notational simplicity, we drop the turn index and use y_t for a generic response token. Both PPO and GRPO optimize this objective via a clipped token-level surrogate loss that shares the same general form:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(y_t, \hat{A}_t) \in \mathcal{D}} w_t \cdot \min\Big(\rho_t(\theta)\hat{A}_t,\ \mathrm{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_t\Big), \tag{2}$$

where $\rho_t(\theta) = \pi_\theta(y_t \mid c_t)/\pi_{\theta_{\mathrm{old}}}(y_t \mid c_t)$ is the importance ratio between the current and old policy, ϵ is the clipping threshold, and $\hat{A}_t$ is the per-token advantage. Here, c_t denotes all tokens preceding y_t, to distinguish it from the environment state s_k, and w_t is a per-token weight.

PPO estimates $\hat{A}_t$ via Generalized Advantage Estimation (GAE) [21] with a learned value function, providing a distinct advantage for each token position. GRPO eliminates the critic and instead computes a single trajectory-level advantage:

$$\hat{A}_\tau = \frac{R(\tau) - \mathrm{mean}(R(\tau))}{\mathrm{std}(R(\tau))}, \tag{3}$$

where the mean and standard deviation are computed over all trajectories sampled from the same prompt. Under GRPO, every token in a trajectory shares the same advantage: $\hat{A}_t = \hat{A}_\tau$ for all t in τ. In both algorithms, the common practice is to set $w_t = 1$ for all response tokens, which corresponds to the uniform credit assignment that we critically challenge in this paper. Complete descriptions for algorithm-specific objectives and implementation details are deferred to Appendix B.

3.2 Frozen-Reference Token Energy

Recent work [7, 33] establishes an equivalence between neural classifiers and energy-based models (EBMs). Given a classifier that maps inputs to C-dimensional logits $h(x) \in \mathbb{R}^C$, the classifier-EBM equivalence is established by setting the energy of an input-label pair as $E(x, y) = -h(x)[y]$. Marginalizing over labels yields a free energy $E(x) = -\log \sum_c \exp(h(x)[c])$, which captures the uncertainty of the model for a given input [12]: lower energy indicates a confident, peaked prediction, while higher energy indicates uncertainty across multiple plausible outputs.

We extend this idea to autoregressive language models, which predict a distribution over the vocabulary at each position. At each token position t, a frozen reference model produces next-token logits $f_t^{\mathrm{ref}} \in \mathbb{R}^{|V|}$ over vocabulary V, conditioned on the preceding context. We define the **token-level energy** as:

$$E_t = -\log \sum_{v \in V} \exp\big(f_{t,v}^{\mathrm{ref}}\big). \tag{4}$$

A lower E_t corresponds to higher confidence under the reference model, while a higher E_t indicates greater uncertainty about the next token. This signal is well-suited for diagnosing agentic RL for two reasons. First, because the reference model is frozen, E_t remains stable throughout training, unlike policy-derived quantities such as entropy or confidence that drift as the policy evolves. Second, computing E_t requires only a single forward pass, making it cheaper than Monte Carlo estimates that require repeated rollouts [23]. We use token-level energy to analyze where the reward-predictive signal concentrates within agent trajectories, and this analysis motivates the intervention in Sec. 4.

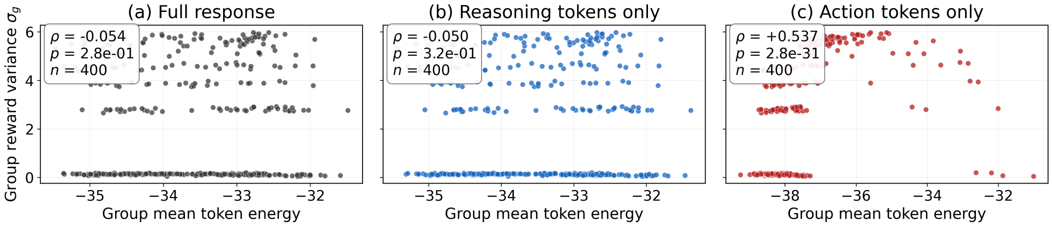


Figure 2: **Detailed energy–reward diagnostic behind Figure 1.** We measure the Spearman correlation between mean frozen-reference token energy and group reward variance σ_g on Sokoban 3B over three token subsets. Only the action-token energy shows a significant positive correlation ($\rho = +0.537$, $p < 10^{-30}$); full-response and reasoning-only energy are indistinguishable from noise.

4 Token Credit Assignment

We now turn the Action Bottleneck diagnosis into a token-level weighting mechanism that aims to address two issues: 1) reasoning tokens dominate the trajectory, and 2) training signals are not uniformly distributed across action tokens. Table 1 reveals the imbalance: across environments and model scales, action tokens account for less than 16% of generated tokens in a trajectory. Given this, uniform credit assignment would concentrate most gradient mass on reasoning spans.

To further validate the results in Figure 1 across different cases, we examine where the training signals live within a trajectory. For each prompt in the training set, we run G independent rollouts under the current policy, each producing a complete multi-turn trajectory and its episodic reward $R(\tau)$. We refer to the G trajectories sampled from the same prompt as a *trajectory group*, with group reward variance $\sigma_g = \mathrm{Var}(R(\tau^{(1)}), \ldots, R(\tau^{(G)}))$. A high σ_g means the policy sometimes succeeds and sometimes fails on this prompt, so the prompt provides useful learning signals. For each trajectory group, we then compute the mean frozen-reference token energy over three token subsets: the full response, reasoning tokens only, and action tokens only. We measure the Spearman correlation between the mean energy of each subset and σ_g across all trajectory groups. Figure 2 shows the result on Sokoban 3B. We find that action-only energy correlates with σ_g, with $\rho = +0.537$ and $p < 10^{-30}$, while full-response and reasoning-only energy are indistinguishable from random noise. This shows that the training signals captured by frozen-reference energy largely stem from action tokens instead of reasoning tokens that dominate in the trajectory.

The observation motivates a token-level reweighting scheme that proceeds in two steps. Specifically, we first use the explicit `<think>`/`<answer>` boundary to reallocate gradient mass from reasoning tokens towards environment-facing action tokens. We then use frozen-reference energy within the action span to prioritize positions where the reference model is more uncertain.

4.1 Reasoning Downweighting and Energy-Based Action Prioritization

For each response token belonging to either a reasoning span $\mathcal{T}_{\text{think}}$ or an action span $\mathcal{T}_{\text{action}}$, as identified by the `<think>` and `<answer>` tags, we assign a weight:

$$w_t = \begin{cases} \alpha, & t \in \mathcal{T}_{\text{think}}, \\ 1 + \beta \ \mathrm{sigmoid}\left(\dfrac{E_t - \mu_E}{\sigma_E}\right), & t \in \mathcal{T}_{\text{action}}, \end{cases} \tag{5}$$

where $\alpha \in [0, 1]$ controls how much gradient is retained on reasoning tokens, $\beta \geq 0$ controls how strongly energy modulates the weights on action tokens, and E_t is the frozen-reference token energy from Eq. 4. μ_E and σ_E denote the mean and standard deviation of E_t over all action tokens in the current training batch, respectively. The sigmoid function is applied to a batch-normalized action-token energy score, making energy modulation depend on relative uncertainty within the action-token distribution rather than on the raw energy scale.

When setting $\alpha = 1$ and $\beta = 0$, which gives $w_t = 1$ for all tokens, the reweighting mechanism described by Eq. 5 reduces to standard uniform credit assignment. Our proposed approach uses $\alpha < 1$ and $\beta > 0$, which simultaneously downweights reasoning tokens and applies energy-based

Table 2: PPO final-step results across four environments at two capability levels. Success rate (%) is reported at step 200 for Sokoban, FrozenLake, and Sudoku, and at step 100 for WebShop. WebShop reports both purchase rate and strict success. Peak values are reported in Table 5. Best results are **bolded**.

Method	**Sokoban**	**FrozenLake**	**Sudoku**	**WebShop$_{\text{purchase}}$**	**WebShop$_{\text{strict}}$**
	3B	3B	1.5B	3B	3B
Uniform	26.2	16.6	81.2	96.9	22.7
ActFocus	**37.5** (+11.3)	**27.0** (+10.4)	**95.7** (+14.5)	**99.2** (+2.3)	**36.7** (+14.0)
	0.5B	0.5B	0.5B	1.5B	1.5B
Uniform	12.5	5.1	16.2	94.9	9.8
ActFocus	**34.0** (+21.5)	**15.6** (+10.5)	**81.4** (+65.2)	**98.8** (+3.9)	**26.6** (+16.8)

redistribution within the action span. The former simply shifts the gradient mass towards informative action tokens, while the latter additionally prioritizes the action tokens for which the model is more uncertain. In our experiments, we compare standard UNIFORM credit assignment against our proposed token reweighting approach, which we refer to as ACTFOCUS.

4.2 Modified Objective

We incorporate the token-level credits by replacing the averaging aggregation in Eq. 2 with a weighted sum:

$$\mathcal{L}(\theta) = -\frac{1}{\sum_t w_t} \sum_t w_t \, \min\Big(\rho_t(\theta)\,\hat{A}_t,\ \mathrm{clip}\big(\rho_t(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\hat{A}_t\Big), \tag{6}$$

where w_t is given by Eq. 5. Tokens with larger w_t contribute more to the policy-gradient update, while tokens with smaller w_t contribute less. Importantly, this modification does not change how the advantage is computed. For PPO, $\hat{A}_t$ still comes from the original critic-based estimator; for GRPO, it still comes from the group-normalized trajectory return. Thus, the method only changes how the existing advantage signal is distributed across tokens.

We normalize by $\sum_t w_t$ so that changing α or β does not simply make the whole loss larger or smaller. Instead, α and β control where the gradient mass goes: decreasing α reduces the contribution of reasoning tokens, while increasing β gives more weight to high-energy action tokens.

Ignoring PPO clipping for clarity, the weighted objective separates the update into a reasoning component and an action component:

$$\nabla_\theta \mathcal{L} \;=\; -\frac{1}{\sum_t w_t} \left[\underbrace{\alpha \sum_{t \in \mathcal{T}_{\text{think}}} \hat{A}_t \nabla_\theta \log \pi_\theta(y_t \mid c_t)}_{\text{reasoning component}} + \underbrace{\sum_{t \in \mathcal{T}_{\text{action}}} w_t \hat{A}_t \nabla_\theta \log \pi_\theta(y_t \mid c_t)}_{\text{action component}} \right]. \tag{7}$$

The two terms correspond directly to the two steps introduced at the start of this section. Reducing α reallocates gradient mass from reasoning spans to action spans. Positive β then redistributes mass within the action span, giving larger weight to high-energy tokens where the frozen reference model is more uncertain.

5 Experiments

We evaluate our proposed method on four multi-turn environments spanning distinct reasoning tasks: **Sokoban**, a spatial planning task where the agent pushes boxes onto targets on a grid; **FrozenLake**, a navigation task on a stochastic slippery surface; **Sudoku 4×4**, a constraint-satisfaction task requiring logical deduction; and **WebShop** [34], an e-commerce task involving product search, evaluation, and purchase. For WebShop, we report both purchase rate and strict success: purchase rate measures whether the agent completes a purchase, while strict success additionally requires the purchased item to match the user instruction. We use the **Qwen2.5-Instruct family** [18] throughout all experiments.

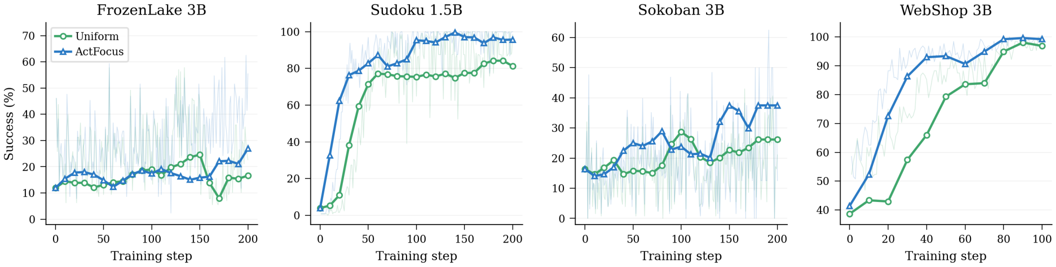


Figure 3: PPO success rate across training steps on FrozenLake 3B, Sudoku 1.5B, Sokoban 3B, and WebShop 3B. Shaded curves denote per-step training success.

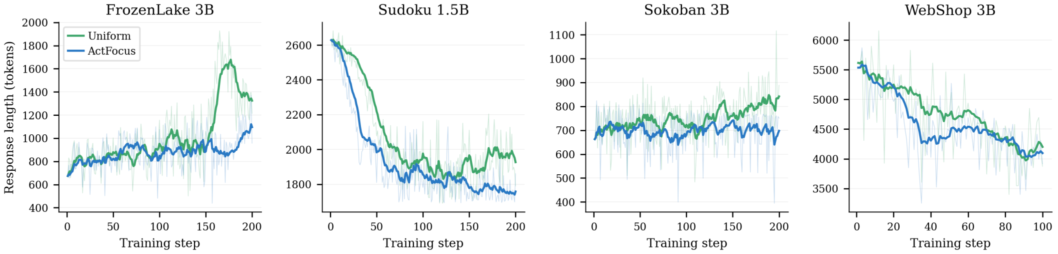


Figure 4: PPO response length across training steps on FrozenLake 3B, Sudoku 1.5B, Sokoban 3B, and WebShop 3B. Shaded curves denote per-step response length.

For each environment, we use two model scales to test whether the Action Bottleneck and reweighting benefits persist across different base-model capabilities. We train each model with both PPO [20] and GRPO [22], built on veRL following StarPO-S configuration [29]. We compare two token-weighting schemes: UNIFORM applies standard equal credit and ACTFOCUS applies the full structure-aware weighting rule in Eq. 5. Detailed dataset specifications, rollout settings, and training hyperparameters are provided in Appendix C.

5.1 Main Results

All main results in Table 2 use a single fixed hyperparameter setting across all four environments and both model scales. We deliberately avoid per-environment tuning to test whether the Action Bottleneck correction generalizes without configuration-specific calibration. Ablations on the choice of α and β are deferred to Sec. 5.4 and Sec. 5.5.

ActFocus improves across all PPO settings. Table 2 shows that ACTFOCUS outperforms UNIFORM in every tested configuration, across four diverse environments and two model scales. The consistent improvement suggests that the gains stem from correcting a shared structural imbalance in agent trajectories rather than fitting to any specific task or model scale.

ActFocus helps regardless of whether Uniform is learning. On Sudoku 0.5B, ACTFOCUS raises success from 16.2% to 81.4%, showing that credit reweighting can unlock learning when uniform assignment delivers little usable signal. Even when UNIFORM already learns nontrivial policies, ACTFOCUS still improves performance by 11.3 points on Sokoban 3B, 10.4 points on FrozenLake 3B, and 14.5 points on Sudoku 1.5B. These results suggest that uniform credit suffers from the same structural problem across regimes: it spreads the learning signal over many reasoning tokens, leaving too little credit for the action tokens that determine the outcome.

ActFocus improves decision quality beyond task completion. On WebShop, UNIFORM already achieves a high purchase rate, so there is limited room to improve the purchase-rate metric. The more informative signal is strict success, where ACTFOCUS improves from 22.7% to 36.7% on WebShop 3B and from 9.8% to 26.6% on WebShop 1.5B. The strict success gain shows that the method improves the quality of the selected product, not merely the ability to finish an episode. This matches the role of frozen-reference energy in Sec. 4: energy identifies uncertain action positions, and ACTFOCUS assigns them larger credit during training.

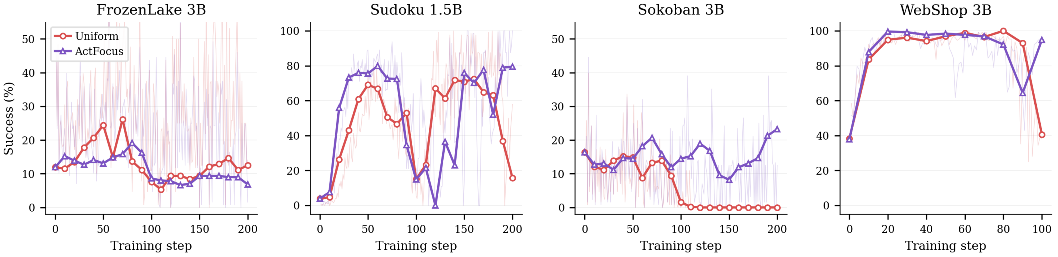


Figure 5: GRPO success rate across training steps on FrozenLake 3B, Sudoku 1.5B, Sokoban 3B, and WebShop 3B. Shaded curves denote per-step training success.

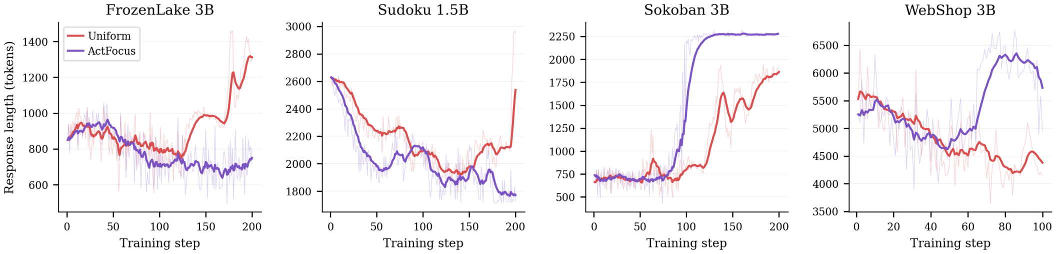


Figure 6: GRPO response length across training steps on FrozenLake 3B, Sudoku 1.5B, Sokoban 3B, and WebShop 3B. Shaded curves denote per-step response length.

5.2 Training Dynamics and Response Efficiency

ActFocus also improves training dynamics. Figure 3 shows that ActFocus improves PPO training dynamics across all four configurations, but the form of improvement differs by environment. On Sudoku 1.5B and WebShop 3B, ActFocus reaches high success earlier than Uniform, indicating faster learning. On Sokoban 3B, the two methods are closer in the early stages, but ActFocus continues to improve in later steps, while Uniform improves more slowly, so the performance gap widens over training. FrozenLake 3B is noisier because of stochastic transitions, but ActFocus still reaches a better final result. Overall, these curves show that the gains in Table 2 are reflected in the training process, rather than appearing only at the final checkpoint.

Gains are not caused by longer responses. Figure 4 shows that ActFocus produces shorter or more stable responses than Uniform across all configurations. On FrozenLake 3B and Sokoban 3B, Uniform response length grows throughout training while ActFocus keeps it controlled. On Sudoku 1.5B and WebShop 3B, both methods shorten responses over training, but ActFocus does so earlier and converges to shorter final lengths. Thus, ActFocus improves success without relying on longer reasoning traces, as better credit assignment directs the policy toward action tokens without requiring excessive verbalization.

5.3 Extension to GRPO

We also evaluate whether the same token-weighting rule transfers to GRPO. Under GRPO, the critic is removed, and a single trajectory-level advantage is assigned to every token in the same rollout, making token weights the primary source of within-trajectory credit differentiation. This makes GRPO especially sensitive to the Action Bottleneck: under uniform credit, reasoning tokens dominate the update even though outcomes are mostly determined by sparse action decisions.

Figure 5 shows that this mismatch often appears as peak-to-final degradation. Across Sokoban 3B, Sudoku 1.5B, and WebShop 3B, Uniform often reaches a useful intermediate policy but fails to preserve it, leading to sharp peak-to-final degradation. ActFocus either prevents the late-stage drop or helps the policy recover from it.

Figure 6 provides a complementary view of stabilization. Under GRPO, response length tracks training stability rather than response efficiency. Noisy length curves coincide with unstable policy behavior, while smooth curves indicate that training has settled into a consistent regime, regardless of whether length is rising, falling, or stable. Stability here does not require shorter responses.

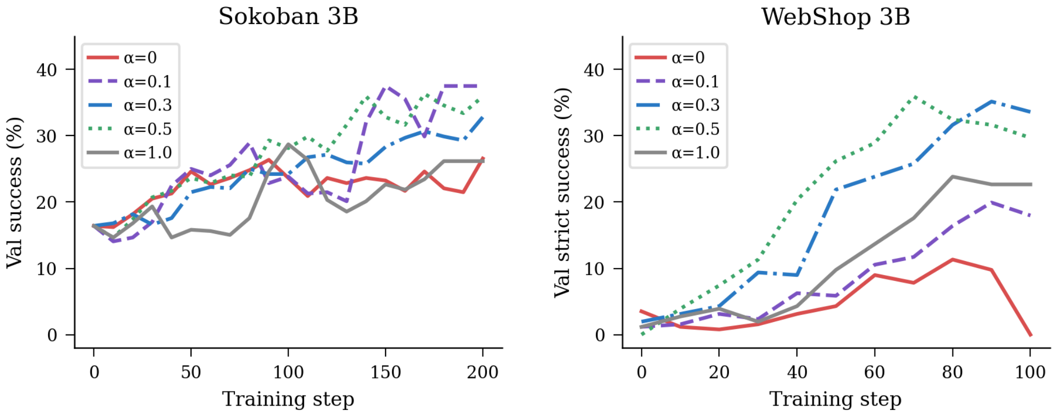


Figure 7: **Effect of α on training dynamics.**

5.4 How Much Reasoning Credit Should Be Retained?

Although ACTFOCUS improves over UNIFORM across all main PPO experiments, the optimal α varies by configuration. The intuition is simple: when the base model already reasons well enough for the task, a smaller α removes redundant reasoning updates and concentrates RL on action decisions; when reasoning still needs refinement, a larger α preserves that gradient signal.

Figure 7 confirms this on two representative environments. On Sokoban 3B, the best result comes from the aggressive setting $\alpha = 0.1$, consistent with a model that already has sufficient spatial reasoning and benefits from focusing updates on action tokens. On WebShop 3B, the optimum shifts to $\alpha = 0.3$, and the extreme $\alpha = 0$ eventually collapses. Unlike Sokoban, WebShop requires ongoing reasoning refinement, such as query reformulation, evidence aggregation, and product comparison, making full reasoning suppression too aggressive.

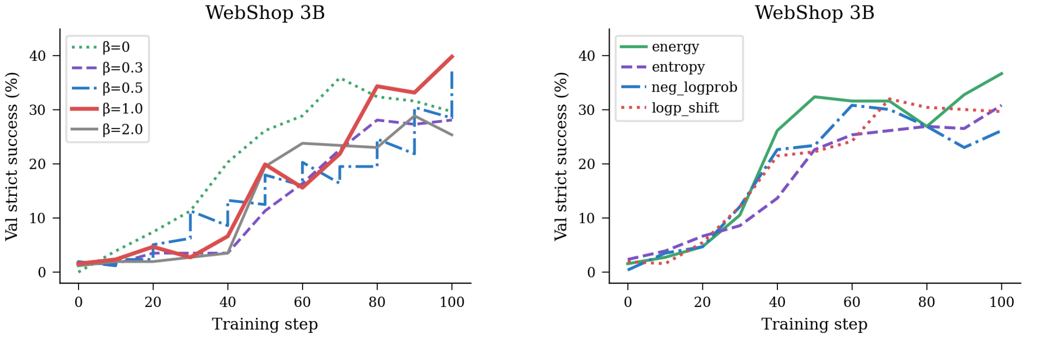
The two extremes illustrate why α requires calibration. At $\alpha = 1.0$, reasoning tokens retain full credit, so energy-based amplification alone cannot overcome the dominance of the reasoning majority. At $\alpha = 0$, reasoning is suppressed entirely, discarding the gradient signal needed for coherent reasoning. The best α sits between these extremes, balancing action focus with reasoning retention.

5.5 How Strongly Should Energy Modulate Action Credit?

With the reasoning-retention parameter α fixed, β controls how strongly energy reshapes credit within the action span. Setting $\beta = 0$ recovers uniform weighting across action tokens, so the action component gain depends only on α, while a larger β progressively concentrates updates on high-energy positions. We sweep $\beta \in \{0, 0.3, 0.5, 1.0, 2.0\}$ on WebShop 3B and train each setting with PPO for 100 steps. As shown in Figure 8a, performance is non-monotonic. Strict success improves from 29.7% at $\beta = 0$ to 39.8% at $\beta = 1.0$, but drops to 25.4% at $\beta = 2.0$, falling below the $\beta = 0$ baseline. This indicates that energy modulation must be strong enough to differentiate uncertain action tokens from confident ones, since treating all action tokens equally fails to exploit the signal energy carries about which decisions are most informative. At the same time, β cannot be too large: when the sigmoid-based weighting becomes too peaked, gradient mass collapses onto a small number of high-energy outliers and leaves the remaining action tokens under-trained.

5.6 Why Energy? A Comparison Against Alternative Signals

A natural question is whether the gains from ACTFOCUS are specific to energy or an artifact of the reweighting architecture. To test this, we replace energy E_t with three alternative signals while keeping all other parameters fixed on WebShop 3B: entropy, policy NLL, and the log-likelihood difference. Figure 8b shows that energy reaches 36.7% strict success, outperforming entropy (30.9%), log-likelihood difference (29.4%), and policy NLL (26.2%). The gap traces back to the properties motivated in Sec. 4: policy-derived signals such as policy NLL and log-likelihood difference shift as the policy updates, making the weighting target a moving one during training, while frozen-reference energy remains stable and retains magnitude information that the frozen-reference entropy discards after normalization. A detailed comparison is included in Appendix C.3.



(a) **Effect of β.** Sweeping β on WebShop 3B: strict success peaks at β=1.0 (39.8%) and degrades at both extremes.

(b) **Signal comparison.** Replacing energy with alternative signals: energy outperforms entropy, log-likelihood difference, and policy NLL.

Figure 8: Ablation studies on the reward-shaping design.

6 Conclusion

In this work, we identify the *Action Bottleneck* as a critical failure mode in agentic reinforcement learning. To address this, we propose a simple structure-aware token reweighting scheme with energy-based redistribution, which redirects gradient mass toward action-relevant decisions. Results across four environments show that this intervention improves learning effectiveness and training stability. Our findings highlight the Action Bottleneck as a central obstacle in agentic RL and show that correcting it provides a principled pathway toward more effective training. Since ACTFOCUS delivers these gains at comparable computational cost to standard PPO and GRPO, it serves as a practical replacement for the uniform token-averaged objective in existing agentic RL pipelines.

References

[1] Marwa Abdulhai, Isadora White, Charlie Snell, Charles Sun, Joey Hong, Yuexiang Zhai, Kelvin Xu, and Sergey Levine. Lmrl gym: Benchmarks for multi-turn reinforcement learning with language models, 2023. URL https://arxiv.org/abs/2311.18232.

[2] Alex J. Chan, Hao Sun, Samuel Holt, and Mihaela van der Schaar. Dense reward for free in reinforcement learning from human feedback, 2024. URL https://arxiv.org/abs/2402.00782.

[3] Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen Koltun, and Philipp Krähenbühl. Reinforcement learning for long-horizon interactive llm agents, 2025. URL https://arxiv.org/abs/2502.01600.

[4] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards, 2025. URL https://arxiv.org/abs/2502.01456.

[5] Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models, 2025. URL https://arxiv.org/abs/2505.22617.

[6] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training, 2025. URL https://arxiv.org/abs/2505.10978.

[7] Will Grathwohl, Kuan-Chieh Wang, Jörn-Henrik Jacobsen, David Duvenaud, Mohammad Norouzi, and Kevin Swersky. Your classifier is secretly an energy based model and you should treat it like one, 2020. URL https://arxiv.org/abs/1912.03263.

[8] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.

[9] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

[10] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Refining credit assignment in rl training of llms, 2025. URL https://arxiv.org/abs/2410.01679.

[11] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. Llms get lost in multi-turn conversation, 2025. URL https://arxiv.org/abs/2505.06120.

[12] Weitang Liu, Xiaoyun Wang, John Owens, and Yixuan Li. Energy-based out-of-distribution detection. *Advances in neural information processing systems*, 33:21464–21475, 2020.

[13] Xiaoqian Liu, Ke Wang, Yuchuan Wu, Fei Huang, Yongbin Li, Junge Zhang, and Jianbin Jiao. Agentic reinforcement learning with implicit step rewards, 2025. URL https://arxiv.org/abs/2509.19199.

[14] Zheng Liu, Mengjie Liu, Siwei Wen, Mengzhang Cai, Bin Cui, Conghui He, and Wentao Zhang. From uniform to heterogeneous: Tailoring policy optimization to every token's nature, 2025. URL https://arxiv.org/abs/2509.16591.

[15] Chiyu Ma, Shuo Yang, Kexin Huang, Jinda Lu, Haoming Meng, Shangshang Wang, Bolin Ding, Soroush Vosoughi, Guoyin Wang, and Jingren Zhou. Fipo: Eliciting deep reasoning with future-kl influenced policy optimization, 2026. URL https://arxiv.org/abs/2603.19835.

[16] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL https://arxiv.org/abs/2203.02155.

[17] Eduardo Pignatelli, Johan Ferret, Matthieu Geist, Thomas Mesnard, Hado van Hasselt, Olivier Pietquin, and Laura Toni. A survey of temporal credit assignment in deep reinforcement learning, 2024. URL https://arxiv.org/abs/2312.01072.

[18] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL https://arxiv.org/abs/2412.15115.

[19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

[20] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

[21] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018. URL https://arxiv.org/abs/1506.02438.

[22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

[23] Leyang Shen, Yang Zhang, Chun Kai Ling, Xiaoyan Zhao, and Tat-Seng Chua. Carl: Focusing agentic reinforcement learning on critical actions, 2026. URL https://arxiv.org/abs/2512.04949.

[24] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[25] Hongze Tan, Zihan Wang, Jianfei Pan, Jinghao Lin, Hao Wang, Yifan Wu, Tao Chen, Zhihang Zheng, Zhihao Tang, and Haihua Yang. Gtpo and grpo-s: Token and sequence-level reward shaping with policy entropy, 2026. URL https://arxiv.org/abs/2508.04349.

[26] Jean Vassoyan, Nathanaël Beau, and Roman Plaud. Ignore the KL penalty! boosting exploration on critical tokens to enhance RL fine-tuning. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6123–6133, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.340. URL https://aclanthology.org/2025.findings-naacl.340/.

[27] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

[28] Xiaoxuan Wang, Han Zhang, Haixin Wang, Yidan Shi, Ruoyan Li, Kaiqiao Han, Chenyi Tong, Haoran Deng, Renliang Sun, Alexander Taylor, Yanqiao Zhu, Jason Cong, Yizhou Sun, and Wei Wang. Arlarena: A unified framework for stable agentic reinforcement learning, 2026. URL https://arxiv.org/abs/2602.21534.

[29] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning, 2025. URL https://arxiv.org/abs/2504.20073.

[30] Muning Wen, Ziyu Wan, Weinan Zhang, Jun Wang, and Ying Wen. Reinforcing language agents via policy optimization with action decomposition, 2024. URL https://arxiv.org/abs/2405.15821.

[31] Georg Wölflein, Dyke Ferber, Daniel Truhn, Ognjen Arandjelović, and Jakob Nikolas Kather. Llm agents making agent tools, 2025. URL https://arxiv.org/abs/2502.11705.

[32] Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, Wei He, Yiwen Ding, Guanyu Li, Zehui Chen, Zhengyin Du, Xuesong Yao, Yufei Xu, Jiecao Chen, Tao Gui, Zuxuan Wu, Qi Zhang, Xuanjing Huang, and Yu-Gang Jiang. Agentgym-rl: Training llm agents for long-horizon decision making through multi-turn reinforcement learning, 2025. URL https://arxiv.org/abs/2509.08755.

[33] Jianwen Xie, Yang Lu, Song-Chun Zhu, and Yingnian Wu. A theory of generative convnet. In *International conference on machine learning*, pages 2635–2644. PMLR, 2016.

[34] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023. URL https://arxiv.org/abs/2207.01206.

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

[36] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan

Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL https://arxiv.org/abs/2503.14476.

[37] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

A Notation

Table 3 summarizes the notation used throughout the paper. Symbols are grouped by category for ease of reference.

Table 3: Summary of notation used in the paper.

Symbol	**Meaning**
Environment and trajectory	
$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R \rangle$	Multi-turn agentic MDP
$s_k \in \mathcal{S}$	State at turn k (interaction history)
$a_k \in \mathcal{A}$	Environment-facing action at turn k
o_k	Policy response at turn k (reasoning + action)
$\tau = (s_1, o_1, \ldots, s_K, o_K)$	Multi-turn trajectory
K	Total number of turns in trajectory τ
T	Total number of response tokens in trajectory τ
$R(\tau)$	Terminal episodic reward
r_t	Per-token reward; $r_t = R(\tau)$ if $t = T$, else 0
Policy and tokens	
π_θ	Trained policy with parameters θ
$\pi_{\theta_{\text{old}}}$	Rollout policy (snapshot used for sampling)
π_{ref}	Frozen reference policy (KL anchor and energy source)
y_t	Generic response token at position t
c_t	Context preceding y_t (state, prior turns, prior tokens)
$\mathcal{T}_{\text{think}}$	Set of token positions inside `<think>` spans
$\mathcal{T}_{\text{action}}$	Set of token positions inside `<answer>` spans
Policy optimization	
$J(\theta)$	Expected episodic return objective
$\rho_t(\theta)$	Token-level importance ratio $\pi_\theta(y_t \mid c_t)/\pi_{\theta_{\text{old}}}(y_t \mid c_t)$
$\hat{A}_t$	Per-token advantage estimate
$\hat{A}_\tau$	Trajectory-level (GRPO) advantage
V_ϕ	Token-level value function used by PPO+GAE
ϵ	PPO clipping threshold
(γ, λ)	Discount factor and GAE coefficient
G	Number of trajectories per trajectory group (GRPO)
$\beta_{\text{KL}}, \beta_{\text{ent}}$	KL and entropy regularization coefficients
Token energy and reweighting	
f_t^{ref}	Frozen-reference next-token logits at position t
E_t	Frozen-reference token energy at position t
μ_E, σ_E	Mean and std of E_t over action tokens in a batch
w_t	Per-token weight in the structure-aware loss
α	Reasoning-retention coefficient (w_t on `<think>` tokens)
β	Action component energy modulation strength
σ_g	Reward variance within a trajectory group

B Reinforcement Learning Background

We follow the notation of Sec. 3: y_t denotes a generic response token, c_t denotes the context consisting of all tokens preceding y_t, including state observations and prior turns within the trajectory, and the token-level importance ratio between the current policy and the rollout policy is:

$$\rho_t(\theta) = \frac{\pi_\theta(y_t \mid c_t)}{\pi_{\theta_{\text{old}}}(y_t \mid c_t)}. \tag{8}$$

Throughout this appendix, T denotes the total number of response tokens in a trajectory τ.

General objective. The agentic RL objective maximizes expected episodic return over multi-turn trajectories:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\big[R(\tau)\big], \tag{9}$$

where $\tau = (s_1, o_1, \ldots, s_K, o_K)$ is a K-turn trajectory and $R(\tau)$ is the terminal episodic reward defined in Sec. 3. The corresponding policy gradient is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=1}^{T} \hat{A}_t \, \nabla_\theta \log \pi_\theta(y_t \mid c_t)\right], \tag{10}$$

which distributes credit across all response tokens y_t in τ via a per-token advantage estimate $\hat{A}_t$. PPO and GRPO differ primarily in how $\hat{A}_t$ is constructed.

Proximal Policy Optimization (PPO). PPO [20] stabilizes the policy update through a clipped surrogate objective combined with a learned token-level value function V_ϕ, which estimates the expected return-to-go from context c_t. The token-level surrogate loss is:

$$\mathcal{L}_{\text{PPO}}(\theta) = -\mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\hat{A}_t,\ \text{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_t\Big)\Big], \tag{11}$$

where ϵ is the clipping threshold. The advantage $\hat{A}_t$ is computed via Generalized Advantage Estimation (GAE) [21]:

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \delta_t = r_t + \gamma V_\phi(c_{t+1}) - V_\phi(c_t), \tag{12}$$

where γ and λ control the bias and variance of the advantage estimate. Following standard practice in LLM RL, the episodic reward $R(\tau)$ is delivered at the final response token of the trajectory: $r_t = R(\tau)$ if $t = T$, and $r_t = 0$ otherwise. The value function V_ϕ is trained jointly with the policy by regressing toward the empirical return-to-go.

Group Relative Policy Optimization (GRPO). GRPO [22] removes the value function by sampling a group of G trajectories $\{\tau^{(i)}\}_{i=1}^{G}$ from the same prompt and computing a group-normalized trajectory advantage:

$$\hat{A}_{\tau^{(i)}} = \frac{R(\tau^{(i)}) - \text{mean}\big(\{R(\tau^{(j)})\}_{j=1}^{G}\big)}{\text{std}\big(\{R(\tau^{(j)})\}_{j=1}^{G}\big)}, \tag{13}$$

which is the explicit group-level form of $\hat{A}_\tau$ in Eq. 3. Because this advantage is defined at the trajectory level, it is broadcast to every token within the trajectory: $\hat{A}_t = \hat{A}_{\tau^{(i)}}$ for all tokens y_t in $\tau^{(i)}$. The clipped surrogate then takes the form:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G}\sum_{i=1}^{G} \mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\hat{A}_{\tau^{(i)}},\ \text{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_{\tau^{(i)}}\Big)\Big]. \tag{14}$$

GRPO reduces memory and computation by removing the critic, but its uniform broadcasting of $\hat{A}_\tau$ across all tokens makes it particularly sensitive to the Action Bottleneck studied in this work.

Auxiliary regularizers. Both PPO and GRPO are typically augmented with two regularizers. A KL term against a reference policy π_{ref} controls how far the trained policy drifts from its initialization:

$$\mathcal{L}_{\text{KL}}(\theta) = \beta_{\text{KL}} \, \mathbb{E}_t\big[D_{\text{KL}}\big(\pi_\theta(\cdot \mid c_t) \,\|\, \pi_{\text{ref}}(\cdot \mid c_t)\big)\big], \tag{15}$$

and an entropy bonus $\mathcal{L}_{\text{ent}}(\theta) = -\beta_{\text{ent}} \, \mathbb{E}_t\big[H\big(\pi_\theta(\cdot \mid c_t)\big)\big]$ encourages exploration during early training. The total objective is $\mathcal{L}_{\text{total}} = \mathcal{L}(\theta) + \mathcal{L}_{\text{KL}}(\theta) + \mathcal{L}_{\text{ent}}(\theta)$, where $\mathcal{L}(\theta)$ is the surrogate loss of either PPO or GRPO. In practice, the same frozen base model serves both as the reference policy π_{ref} for KL regularization and as the source of the frozen-reference logits used to compute token energy in Eq. 4. The specific values of β_{KL} and β_{ent} used in our experiments are listed in Appendix C.

Relation to our weighting scheme. The structure-aware reweighting in Eq. 6 is orthogonal to both algorithms' choice of $\hat{A}_t$. Under PPO, $\hat{A}_t$ remains the GAE estimate; under GRPO, it remains the group-normalized trajectory advantage. ACTFOCUS only changes the per-token weight w_t that multiplies the surrogate, so it composes with PPO and GRPO without modifying their advantage estimators or auxiliary regularizers.

C Experimental Details

C.1 Task Environment Details

We evaluate our method on four multi-turn environments that span a broad range of reasoning demands and interaction structures. Sokoban, FrozenLake, and Sudoku are compact symbolic environments with fully specified dynamics, making them well-suited for controlled analysis of reinforcement learning behavior and token-level credit assignment. Although all three are synthetic, they stress different forms of difficulty: irreversible planning in Sokoban, stochastic transitions in FrozenLake, and structured constraint reasoning in Sudoku. To complement these controlled settings, we also include WebShop [34], a more realistic environment in which the agent must interpret natural-language instructions, navigate a semi-structured interface, and complete a purchase through a sequence of grounded actions. Taken together, these environments allow us to study the Action Bottleneck under both tightly controlled symbolic tasks and a more open-ended web interaction setting.

Sokoban. Sokoban is a grid-based planning task in which the agent must push boxes onto target cells within a limited action budget. Unlike standard navigation problems, Sokoban is irreversible: boxes can be pushed but not pulled back, so an early mistake can permanently destroy a valid solution path. This makes the environment a useful testbed for long-horizon planning and action precision in multi-turn agent training.

FrozenLake. FrozenLake combines goal-directed planning with stochastic transitions. The agent must navigate a grid world to reach the goal while avoiding holes, but actions are executed on slippery tiles and may lead to unintended outcomes. As a result, good performance depends not only on moving toward the goal but also on choosing actions that remain robust under transition uncertainty.

Sudoku. Sudoku evaluates structured reasoning in a sequential decision-making setting. The agent fills a 4×4 grid one move at a time, subject to row, column, and subgrid constraints. Because each valid action depends on a set of coupled logical conditions, the environment places more emphasis on inference and constraint satisfaction than on exploration or long-horizon control.

WebShop. WebShop [34] is a realistic multi-turn web interaction environment in which the agent must satisfy a natural-language shopping request through search, navigation, option selection, and purchase. Compared with the symbolic environments, WebShop introduces richer language grounding, a semi-structured interface, and a less cleanly defined action space. It therefore serves as a complementary setting for studying whether token-level credit assignment mechanisms continue to help when reasoning and action unfold in a more realistic decision-making process.

C.2 Training and Evaluation Settings

We conduct our experiments using the **Qwen2.5-Instruct** family [18] at three scales: 0.5B, 1.5B, and 3B. For each configuration, we train with both PPO [20] and GRPO [22] under two token-weighting variants: UNIFORM and ACTFOCUS. All experiments are built on the RAGEN framework [29], which is implemented on top of veRL[3], and are run on a single node with $4\times$ NVIDIA H100 GPUs. We train for 200 rollout–update iterations on Sokoban, FrozenLake, and Sudoku 4×4, and for 100 iterations on WebShop [34] due to its long-context nature.

Each on-policy update uses grouped rollouts. On Sokoban, FrozenLake, and Sudoku, we use prompt-group size $P = 8$ and sample $G = 16$ trajectories per prompt; on WebShop, we use $P = 16$ and $G = 8$. This yields 128 trajectories per training step before filtering. Agents are allowed at most

---

[3] `https://github.com/volcengine/verl`

2 actions per turn and 10 actions per episode on Sokoban and FrozenLake (5-turn horizon), an 8-turn horizon with a 16-step cap on Sudoku, and a 9-turn horizon with a single action per turn on WebShop. Following StarPO-S [29], we apply reward-variance trajectory selection with filter ratio 0.25 on Sokoban and Sudoku. On WebShop and GRPO-based FrozenLake, we use a filter ratio of 1.0 (no filtering), since environmental stochasticity contaminates within-prompt reward variance as a selection signal. PPO uses a mini-batch size of $E = 32$, with per-GPU micro-batch sizes scaled to memory (1–8, depending on model scale and environment), under FSDP and Ray for distributed training.

Policy optimization uses GAE with $(\gamma, \lambda) = (1.0, 1.0)$, Adam with $(\beta_1, \beta_2) = (0.9, 0.999)$, actor learning rate 1×10^{-6}, and critic learning rate 1×10^{-5}. We use asymmetric clipping $(\epsilon_{\text{low}}, \epsilon_{\text{high}}) = (0.2, 0.28)$ following Yu et al. [36], together with entropy regularization of coefficient 0.001. For Sokoban, FrozenLake, and Sudoku, we drop the KL loss term during training and track KL post hoc, following Yu et al. [36]. For WebShop under GRPO, we add a KL loss with coefficient 0.001 and the k_1 estimator to reduce late-stage collapse, following the vanilla StarPO setting in Wang et al. [29]. We impose a format penalty of -0.1 when the agent fails to produce a valid `<think>/<answer>` structure. Our two token-weighting variants instantiate w_t in Eq. 5: UNIFORM $(\alpha{=}1.0, \beta{=}0)$ and ACTFOCUS $(\alpha{=}0.1, \beta{=}0.5)$. Rollouts are generated with vLLM using `tensor_parallel_size=1` and `enforce_eager`, with a maximum model length of 3600 tokens and 400-token responses for Sokoban, FrozenLake, and Sudoku, and an 8192-token context window for WebShop.

Evaluation. We evaluate every 10 training steps and also before training begins (`val_before_train`). For Sokoban, FrozenLake, and Sudoku, we use a fixed evaluation set of 32 distinct prompts rolled out 16 times each under stochastic decoding, for a total of 512 trajectories per evaluation. For WebShop, we use 256 distinct prompts with one rollout each (256 trajectories). Decoding uses a temperature of 0.5 to measure robustness under sampling. Evaluation episodes are truncated using the same per-environment turn and action budgets as in training. We report evaluation success rate on this fixed evaluation set at step 200 for Sokoban, FrozenLake, and Sudoku, and at step 100 for WebShop.

C.3 Signal Variants for the Energy Ablation

This appendix defines the four action-token signals compared in Sec. 5.6. The goal is to separate the weighting architecture from the choice of signal. All variants use the same weighting pipeline. They differ only in the raw score s_t assigned to each action token.

Common pipeline. For a batch of action-token positions $\mathcal{T}_{\text{action}}$, we first normalize the raw signal s_t within the action tokens:

$$\mu_s = \frac{1}{|\mathcal{T}_{\text{action}}|} \sum_{t \in \mathcal{T}_{\text{action}}} s_t, \qquad \sigma_s = \sqrt{\frac{1}{|\mathcal{T}_{\text{action}}|} \sum_{t \in \mathcal{T}_{\text{action}}} (s_t - \mu_s)^2 + \varepsilon}. \tag{16}$$

We then map the normalized score to $(0, 1)$:

$$\tilde{s}_t = \operatorname{sigmoid}\left(\frac{s_t - \mu_s}{\sigma_s}\right). \tag{17}$$

The final token weight is

$$w_t = \begin{cases} \alpha, & t \in \mathcal{T}_{\text{think}}, \\ 1 + \beta\, \tilde{s}_t, & t \in \mathcal{T}_{\text{action}}. \end{cases} \tag{18}$$

Thus, all four variants use the same α, β, normalization, and sigmoid mapping. Only the raw action-token signal s_t changes.

(1) Frozen-reference energy. Energy is the signal used by ACTFOCUS. It measures uncertainty from the frozen reference model before applying any policy update:

$$s_t^{\text{energy}} = E_t = -\log \sum_{v \in V} \exp\bigl(f_{t,v}^{\text{ref}}\bigr), \tag{19}$$

where $f_{t,v}^{\text{ref}}$ is the frozen-reference logit for vocabulary token v. This matches the token-level energy definition in Eq. 4. Because the reference model is frozen, this score does not move as training progresses.

(2) Frozen-reference entropy. Entropy is another fixed-reference uncertainty signal. It measures how spread out the frozen reference distribution is:

$$s_t^{\text{entropy}} = H\big(\pi_{\text{ref}}(\cdot \mid c_t)\big) = -\sum_{v \in V} p_{t,v}^{\text{ref}} \log p_{t,v}^{\text{ref}}, \qquad p_{t,v}^{\text{ref}} = \frac{\exp(f_{t,v}^{\text{ref}})}{\sum_{u \in V} \exp(f_{t,u}^{\text{ref}})}. \tag{20}$$

Entropy uses the same frozen-reference forward pass as energy. The difference is that entropy is computed after softmax normalization, and therefore captures only the dispersion of the distribution, discarding the absolute logit scale.

(3) Policy NLL. Policy NLL measures how surprising the sampled token is under the rollout policy:

$$s_t^{\text{nll}} = -\log \pi_{\theta_{\text{old}}}(y_t \mid c_t). \tag{21}$$

This signal is cheap because PPO already stores rollout log-probabilities. However, it is policy-dependent: as the policy changes during training, the meaning and scale of this signal also change.

(4) Log-probability shift. Log-probability shift measures how much the rollout policy has moved away from the frozen reference at the sampled token:

$$s_t^{\text{shift}} = \log \pi_{\theta_{\text{old}}}(y_t \mid c_t) - \log \pi_{\text{ref}}(y_t \mid c_t). \tag{22}$$

A large positive value indicates that the rollout policy assigns a higher probability to y_t than the frozen reference does. The log-probabilities are computed from logits without additional temperature scaling. Unlike energy and entropy, this signal partly moves with the policy because it contains $\pi_{\theta_{\text{old}}}$.

Signal stability. Table 4 summarizes whether each raw signal changes as the rollout policy updates. The empirical ranking in Sec. 5.6 matches this stability distinction. Energy and entropy are computed from a fixed model, so they provide stable weighting targets throughout training. Policy NLL and log-probability shift depend on the rollout policy, so their values change as the policy is updated. Among the two frozen-reference signals, energy performs better in our experiments. A useful interpretation is that energy keeps scale information from the reference logits, while entropy keeps only the normalized distribution shape. This makes frozen-reference energy a natural fit for ACTFOCUS: it is stable during training and still preserves confidence information useful for prioritizing uncertain action tokens.

Table 4: Stability of the four token-level signals during training. Frozen-reference signals provide a fixed weighting target, while policy-derived signals move as the policy changes.

Signal	**Source**	**Drifts during training?**
Energy	Frozen reference π_{ref}	No
Entropy	Frozen reference π_{ref}	No
Policy NLL	Rollout policy $\pi_{\theta_{\text{old}}}$	Yes
Log-probability shift	Rollout policy and frozen reference	Partly

D Additional Experimental Results

Table 5: PPO and GRPO peak performance across four environments. Values are success/purchase rate (%); deltas are relative to Uniform. WebShop reports both purchase rate and strict success. Best results are **bolded**.

Alg.	**Method**	**Sokoban**	**FrozenLake**	**Sudoku**	**WebShop$_{\text{purchase}}$**	**WebShop$_{\text{strict}}$**
		3B	3B	1.5B	3B	3B
PPO	Uniform	28.7	24.6	84.2	98.0	23.8
	ActFocus	**37.5** (+8.8)	**27.0** (+2.4)	**99.6** (+15.4)	**99.6** (+1.6)	**30.5** (+6.7)
GRPO	Uniform	17.8	**26.2**	70.9	**100.0**	44.9
	ActFocus	**23.2** (+5.4)	19.1 (-7.1)	**80.9** (+10.0)	99.6 (-0.4)	**51.6** (+6.7)
		0.5B	0.5B	0.5B	1.5B	1.5B
PPO	Uniform	**43.6**	16.2	16.2	94.9	10.9
	ActFocus	40.4 (-3.2)	**20.9** (+4.7)	**82.0** (+65.8)	**99.2** (+4.3)	**27.0** (+16.1)
GRPO	Uniform	20.7	23.8	0.0	99.6	49.2
	ActFocus	**23.4** (+2.7)	**25.2** (+1.4)	0.0 (no learning)	**100.0** (+0.4)	**64.5** (+15.3)

Table 6: GRPO final-step results across four environments at the larger of the two model scales tested per environment. Success rate (%) is reported at step 200 for Sokoban, FrozenLake, and Sudoku, and at step 100 for WebShop. WebShop reports both purchase rate and strict success. Small-scale GRPO results are omitted because uniform credit largely fails to learn at this scale (e.g., Sudoku 0.5B reaches 0.0 in Table 5). Uniform values here reflect the peak-to-final degradation under critic-free training (see Sec. 5.3). Best results are **bolded**.

Method	**Sokoban**	**FrozenLake**	**Sudoku**	**WebShop$_{\text{purchase}}$**	**WebShop$_{\text{strict}}$**
	3B	3B	1.5B	3B	3B
Uniform	0.0	**12.5**	15.8	40.6	14.1
ActFocus	**23.2** (+23.2)	6.8 (-5.7)	**79.5** (+63.7)	**94.9** (+54.3)	**51.6** (+37.5)

E Environment Prompts and Example Rollouts

E.1 Prompt Templates

Across environments, we use the same three-role chat template: a fixed `system` message that describes the environment, a per-turn `user` message containing the current state, and an `assistant` message produced by the policy. The system message is assembled from the per-environment instruction together with an optional grid-symbol vocabulary and, when applicable, an enumeration of admissible actions. The user message follows the same general structure across environments: `(Turn` n`):\nState:\n{state}\nYou have {actions_left} actions left. Always output: <think> [Your thoughts] </think> <answer> [your answer] </answer> with no extra text. Strictly follow this format. Max response length: {max_tokens} words (tokens).`

Below, we show, for each of the four environments used in our experiments, the exact `system` prompt emitted by the training pipeline.

E.1.1 Sokoban (`SimpleSokoban`)

Sokoban System Prompt

```
You're a helpful assistant. You are solving the Sokoban puzzle.
You are the player and you need to push all boxes to targets.
When you are right next to a box, you can push it by moving in
the same direction.
You cannot push a box through a wall, and you cannot pull a box.
The answer should be a sequence of actions, like
<answer>Right || Right || Up</answer>
```

```
The meaning of each symbol in the state is:
#: wall, _: empty, O: target, √: box on target, X: box, P: player,
S: player on target
Your available actions are:
Up, Down, Left, Right
You can make up to 10 actions, separated by the action separator " || "
```

E.1.2 Sudoku (`SimpleSudoku`, 4×4)

Sudoku System Prompt

```
You're a helpful assistant. You are solving a 4x4 Sudoku puzzle.
Fill in the grid so that every row, column, and 2x2 box contains
the numbers 1-4 without repetition.
Initial cells are shown in [brackets] and cannot be modified.
Empty cells are shown as dots (.).
Place numbers one at a time using the format:
<answer>place 3 at row 1 col 2</answer> or <answer>1,2,3</answer>
The environment will provide feedback on valid/invalid moves and
show conflicts if any occur.
```

E.1.3 FrozenLake (`CoordFrozenLake`, 4×4, slippery)

FrozenLake System Prompt

```
You're a helpful assistant. You are solving the FrozenLake puzzle.
The observation includes both a symbol grid and zero-indexed
coordinates for the start, goal, player, and any holes.
Beware that the ice is slippery, so the agent might slide and end
up in an unintended tile.
Respond with a sequence of actions such as
<answer>Left || Up || Up</answer>.

The meaning of each symbol in the state is:
P: player, _: empty, O: hole, G: goal, X: player in hole,
√: player on goal
Your available actions are:
Left, Down, Right, Up
You can make up to 10 actions, separated by the action separator " || "
```

E.1.4 WebShop

The WebShop system prompt contains the task description only; the admissible-action list is inserted into each turn's state string rather than the system prompt itself.

WebShop System Prompt

```
You're a helpful assistant. You are browsing an online shop. Based
on the instruction, buy a product that close to the production
description. You need to search, read the search results, pick a
product, choose the size and color and buy. You should only choose
action from the available actions list provided later. Example
process: I need a gingko light and 20x20 pillow cover that is hand
painted. First search[gingko light 20x20 pillow cover hand painted],
answer format: <answer>search[blanket with fleece throw]</answer>.
```

```
Valid answer is search[<keywords>] or click[<clickable>].
```

E.2 Multi-turn State–Action Templates

Each example below is a verbatim excerpt from an evaluation-time rollout. We preserve the exact `system / user / assistant` role markers, state formatting, and model outputs.

E.2.1 Sokoban

Sokoban Rollout Example (Turns 1–2)

Input:

```
system
You're a helpful assistant. You are solving the Sokoban puzzle.
You are the player and you need to push all boxes to targets.
When you are right next to a box, you can push it by moving in
the same direction.
You cannot push a box through a wall, and you cannot pull a box.
The answer should be a sequence of actions, like
<answer>Right || Right || Up</answer>

The meaning of each symbol in the state is:
#: wall, _: empty, O: target, √: box on target, X: box, P: player,
S: player on target
Your available actions are:
Up, Down, Left, Right
You can make up to 10 actions, separated by the action separator " || "

user

Turn 1:
State:
######
###P_#
###X_#
###__#
###O_#
######
You have 10 actions left. Always output: <think> [Your thoughts]
</think> <answer> [your answer] </answer> with no extra text.
Strictly follow this format. Max response length: 100 words (tokens).
```

Output (Turn 1):

```
<think>The player is right next to a box. The box is not on a target
yet. The target is directly above the player.</think>
<answer>Right || Up</answer>
```

Next User message (Turn 2):

```
user
Reward:
-0.2

Turn 2:
State:
######
###_P#
###X_#
###__#
```

```
###O_#
######
You have 8 actions left. Always output: <think> [Your thoughts]
</think> <answer> [your answer] </answer> with no extra text.
Strictly follow this format. Max response length: 100 words (tokens).
```

Output (Turn 2):

```
<think>After pushing the box right, the player is now right next to
the target. The box is now on the target.</think>
<answer>Up</answer>
```

E.2.2 Sudoku

Sudoku Rollout Example (Turn 1)

Input:

```
system
You’re a helpful assistant. You are solving a 4x4 Sudoku puzzle.
Fill in the grid so that every row, column, and 2x2 box contains
the numbers 1-4 without repetition.
Initial cells are shown in [brackets] and cannot be modified.
Empty cells are shown as dots (.).
Place numbers one at a time using the format:
<answer>place 3 at row 1 col 2</answer> or <answer>1,2,3</answer>
The environment will provide feedback on valid/invalid moves and
show conflicts if any occur.

user

Turn 1:
State:
==================================================
SUDOKU PUZZLE
==================================================
[3][1]| [2][4]
[4] . |  .  .
-------------
 . [3]| [4] .
 . [4]| [3][2]

Legend: [N]=initial cell, N=user-placed, *N*=conflict, .=empty

VALID NUMBERS FOR EMPTY CELLS:
  - (2,2): [2]
  - (2,3): [1]
  - (2,4): [1, 3]
  - (3,1): [1, 2]
  - (3,4): [1]
  - (4,1): [1]

Progress: 10/16 cells filled (10 initial, 0 placed)
Steps: 0/16
You have 10 actions left. Always output: <think> [Your thoughts]
</think> <answer> [your answer] </answer> with no extra text.
Strictly follow this format. Max response length: 150 words (tokens).
```

Output:

```
<think>Considering the initial placement, the only number that can go
in (2,2) is 2.</think>
<answer>2,2,2</answer>
```

E.2.3 FrozenLake

FrozenLake Rollout Example (Turns 1–2)

Input:

```
system
You're a helpful assistant. You are solving the FrozenLake puzzle.
The observation includes both a symbol grid and zero-indexed
coordinates for the start, goal, player, and any holes.
Beware that the ice is slippery, so the agent might slide and end
up in an unintended tile.
Respond with a sequence of actions such as
<answer>Left || Up || Up</answer>.

The meaning of each symbol in the state is:
P: player, _: empty, O: hole, G: goal, X: player in hole,
√: player on goal
Your available actions are:
Left, Down, Right, Up
You can make up to 10 actions, separated by the action separator " || "

user

Turn 1:
State:
Coordinates:
Board size: 4 rows x 4 cols (zero-indexed).
Start: (2, 3)
Goal: (3, 0)
Player: (2, 3)
Holes: (0, 3), (1, 1)
Grid Map:
___O
_O__
___P
G___
You have 10 actions left. Always output: <think> [Your thoughts]
</think> <answer> [your answer] </answer> with no extra text.
Strictly follow this format. Max response length: 120 words (tokens).
```

Output (Turn 1):

```
<think>The player is at position (2, 3) and needs to move towards
the goal at (3, 0). Moving left is the first logical step.</think>
<answer>Left || Left</answer>
```

Next User message (Turn 2):

```
user
Reward:
0

Turn 2:
State:
Coordinates:
Board size: 4 rows x 4 cols (zero-indexed).
```

```
Start: (2, 3)
Goal: (3, 0)
Player: (2, 1)
Holes: (0, 3), (1, 1)
Grid Map:
___O
_O__
_P__
G___
You have 8 actions left. Always output: <think> [Your thoughts]
</think> <answer> [your answer] </answer> with no extra text.
Strictly follow this format. Max response length: 120 words (tokens).
```

Output (Turn 2):

```
<think>After moving left, the player is now at (2, 1). Moving left
again would lead the player into a hole. The next best move is to
move up.</think>
<answer>Up</answer>
```

E.2.4 WebShop

WebShop Rollout Example (Turn 1)

Input:

```
system
You're a helpful assistant. You are browsing an online shop. Based
on the instruction, buy a product that close to the production
description. You need to search, read the search results, pick a
product, choose the size and color and buy. You should only choose
action from the available actions list provided later. Example
process: I need a gingko light and 20x20 pillow cover that is hand
painted. First search[gingko light 20x20 pillow cover hand painted],
answer format: <answer>search[blanket with fleece throw]</answer>.
Valid answer is search[<keywords>] or click[<clickable>].

user

Turn 1:
State:
Instruction: Find me machine wash, wash cold women's fashion hoodies
& sweatshirts for dry clean, tumble dry with color: vintage camo,
and size: large, and price lower than 70.00 dollars.We must buy a
product within 10 actions. It doesn't have to match perfectly with
description.
Search term should not include details like size, color.
Never search for more than 2 times.
Do not be too strict about the description, it's more important to
buy one that is close enough within action limit.
Prioritize click a product in the current page over going to next
page.
Almost never click[next >] for more than 2 times.Almost never
click[< prev] unless you are sure the product is on one of the
previous pages.
If you have less than 3 actions left, just buy the first product you
see in the current page.
If an matching option exists, make sure to click[size] then
```

```
click[color], one at a time, before click[buy now], but don’t have
to if only 1 action left, in that case you just click[buy now].
Never click description.
You must choose from these actions:search[<content>].
You have 9 actions left. Always output: <think> [Your thoughts]
</think> <answer> [your answer] </answer> with no extra text.
Strictly follow this format. Max response length: 200 words (tokens).
```

Output:

```
<think>Based on the instruction, I need to find a machine washable,
wash cold women’s hooded sweatshirt that is vintage camo pattern and
in large size, with a price lower than 70.00 dollars. The search
term should not include size or color.</think>
<answer>search[women hooded sweatshirt vintage camo large machine
wash cold]</answer>
```